Practical Machine Learning

Exercise 3
Spring 2024

Lisa Stafford

February 20, 2024

Abstract

Model selection is a common problem for machine learning architects where one selects the model that does the best job of generalizing and most appropriate for the given problem. Those selecting the model should choose the best balance between under- and over-fitting. When selecting a model, one must examine the value of different predictive methods to identify the one that best fits observable data. There are two main factors when choosing the most appropriate model in machine learning. One is the reason for choosing it, and the other is the model's performance. Reasons for choosing a model should be based on the available data set, and the problem task.

Introduction

In order to learn and determine which models are most effective, we used a wine data set obtained from the University of Irvine [1]. Training is performed on each subset of red and white wine within the data set. Several models are trained implementing libraries directly from scikitlearn [4] and then training and testing performance scores for each model and wine subsets are taken. Model selection determinations can then be made based on the performance scores.

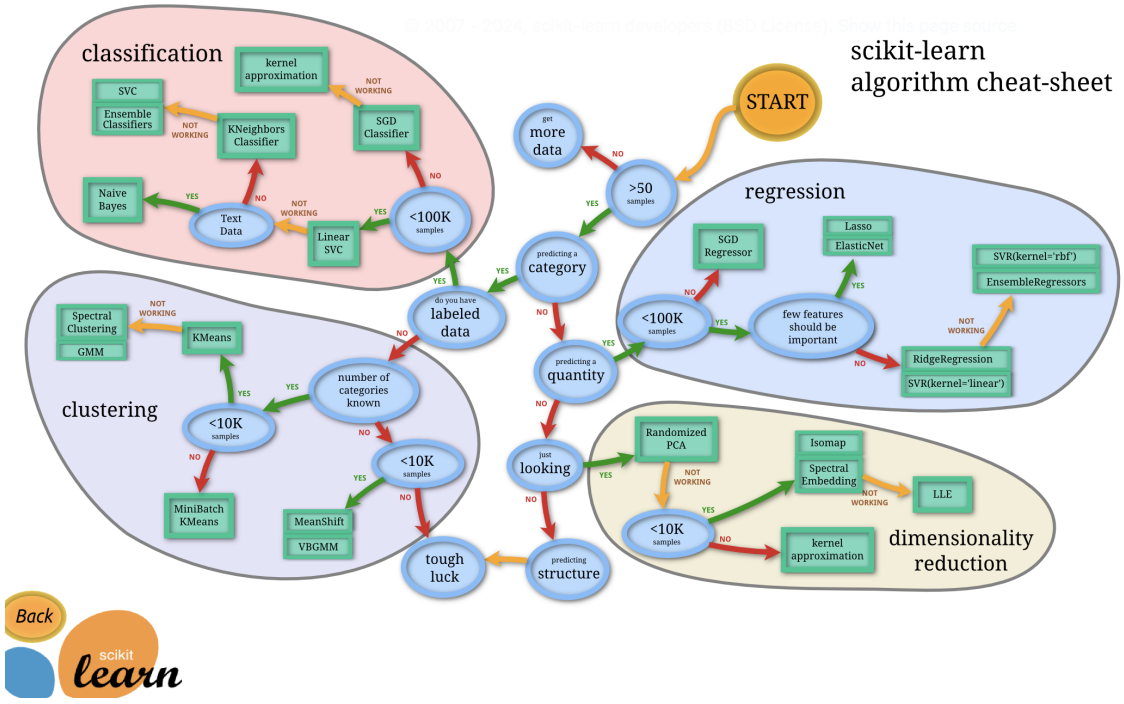
Dataset Description

Our Wine Dataset is actually comprised of two different wine subsets of data. Both are related to variants of [1] "Portuguese Vhinho Verde wine". The data is thereby split in two subsets - one red subset and one white subset. Each subset contains 11 features and 1 label for wine quality. All features are continuous float values. Within the model the "wine quality" labels are recognized as discrete categorical integer values ranging from 3 to 9 for the white wines, and 3 and 8 for the red wines. White wine contains a total of 4898 total data instances, while red wine has a total of 1598 data instances.

Data for both red and white wine datasets is somewhat "pre-cleaned". The feature values are all continuous and both red and white datasets contain no missing data values. However it should be noted that these features are not evenly distributed. The data values are somewhat bell-shaped with a larger number of data instances with labels closer to the mean. As described on the dataset website [1] both "dataset... classes are ordered and []unbalanced] so there are many more normal wines than excellent or poor ones.... [also], both datasets have 11 physiochemical features... and a sensory output label, ('quality')". Because Standard Scaling and Transformation and Log Transformation worked best as a data transformation method in Exercise 2, we use the same code to perform these two specific data cleaning methods on the data again.

Experimental Setup

We are looking at the wine quality dataset, and trying to run a small number of machine learning algorithms on the dataset to determine the best machine learning algorithm for the dataset, where the "best" algorithm may be a set of algorithms.
We know that this particular dataset is supervised and the problem is a classification problem with a known number of categories. Therefore we know that the best ML models will be in that realm. To make an initial decision, this "cheat sheet" decision chart from sklearn [4] was followed:



A decision was made to evaluate a number of popular Classification ML Models/Algorithms, since it was hard to be sure which would be more effective. The machine learning algorithms that fit best with this particular dataset are as follows and therefore we will utilize these listed in our initial ML model selection:

1. Random Forest Classifier
2. Linear Support Vector Machine
3. Classic Support Vector Machine
4. K-Neighbors Classifier
5. Stochastic Gradient Descent Classifier
6. Neural Network using MultiLayer Perceptron

While neural networks are not listed in the above decision chart, they are frequently used when training ML models due to their flexibility for use in a wide variety of ML problems, on different data types, and generally high level of performance. Therefore, they were included in this exercise.

Initially, training was performed on each subset of red and white wine within the data set and data is parsed into 80% for training, and 20% for testing using the standard train-test-split library in scikitlearn, running each train test split 5 times, and averaging results. In an effort to provide more standardized and accurate assessment of the overall model performance, this was updated to use the scikitlearn stratifiedkfold library [4] using 5 folds. All above listed models are trained implementing libraries directly from scikitlearn [4]. While we have a number of different model options, we specify limited hyper-parameters for each model, and instead frequently proceed with training using the default hyper-parameters specified through scikitlearn in most cases.

Next, training and testing performance scores for each model and wine subsets are taken. Listed training and test scores for pre and post scaling is shown from the results of a 5 fold stratefiedkfold

accuracy results, rather than a standard train-test-split. 5 folds were chosen over the usual 10-fold cross validation because using a larger number of folds resulted in a number of warnings from the scikit-learn library due to a lack of data for the less populated classes. Future work in upcoming exercises may look at addressing this issue.

Following the training and testing of models, model selection determinations and evaluation is then made based on these 5 stratified k fold train and test runs to get performance scores of each model.

Results

Machine learning performance results were never very highly performative, but resulted as follows. The highest machine learning test performance was found to be Random Forest models, followed by K-Nearest Neighbors models regardless of training the red or white datasets, or the scaling of data. While the most performative models appeared to be more immune to data scaling and transforms. The following two charts show performance of each mode type on red wine data before and then after performing data scaling. Data scaling is noted to have a significant effect on performance of certain ML model, specifically Logarithmic Support Vector Machine Classification, Stochastic Gradient Descent Classification, and Multilayer Perceptron Classification.

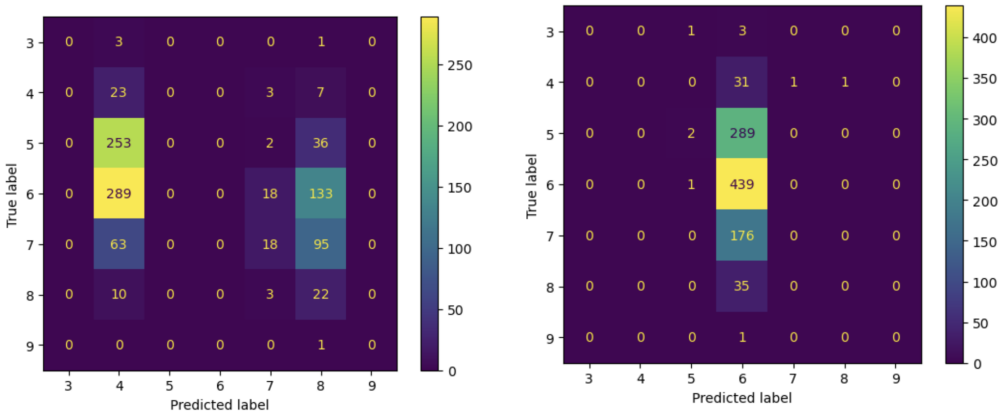
Pre Scaling:

	Model_Type	Wine_Type	Training_Score	Test_Score	Data_Scale
0	Random Forest	red	1.000000	0.683546	none
1	Linear SVC	red	0.578175	0.572864	none
2	Poly SVC	red	0.788147	0.565980	none
3	KNN	red	1.000000	0.609784	none
4	SGD	red	0.374287	0.367147	none
5	MLP	red	0.498434	0.494690	none

Post Scaling:

6	Random Forest	red	1.000000	0.696681	standard
7	Linear SVC	red	0.589275	0.582862	standard
8	Poly SVC	red	0.843809	0.630396	standard
9	KNN	red	1.000000	0.669165	standard
10	SGD	red	0.586617	0.585986	standard
11	MLP	red	0.612725	0.593495	standard

A confusion matrix is generated for each iteration of train/test with each model to "take the pulse" for each method of training and variations between them. It was notable that certain model types such as Linear SVM and SGD resulted in variable performance on models each run. Worst performing models were those that failed to account for outliers and would typically only classify in 2 categories. The further from the median these categories were, the worse the performance of the overall SGD model. Two examples of such failing model confusion matrices are shown for reference below. Future investigatory work may seek to determine why these two models resulted in such bad predictions.



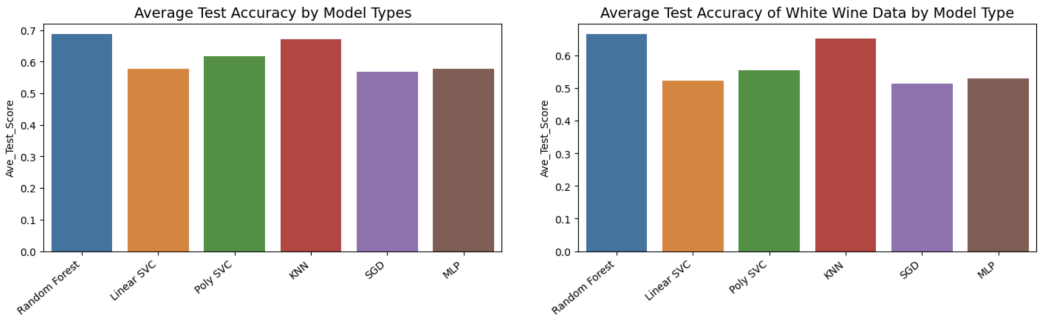
Highest performance for the white wine subset was obtained using a Random Forest Classification Model. The random forest model achieves an average 68.5% accuracy for performance on red wine test data and 66% accuracy on white wine test data. Results are displayed in the chart below with red results on the left and white results on the right.

	Model_Type	PreScaled_Test_Score	Min_Test_Score	Ave_Test_Score	Max_Test_Score	Best_Data_Scale
0	Random Forest	0.696679	0.680429	0.686885	0.691675	standard
1	Linear SVC	0.572864	0.572859	0.576193	0.582862	standard
2	Poly SVC	0.565980	0.606009	0.617474	0.630396	standard
3	KNN	0.609784	0.667927	0.670003	0.672917	standard
4	SGD	0.452147	0.549732	0.567658	0.588503	standard
5	MLP	0.495921	0.535974	0.578283	0.601634	standard

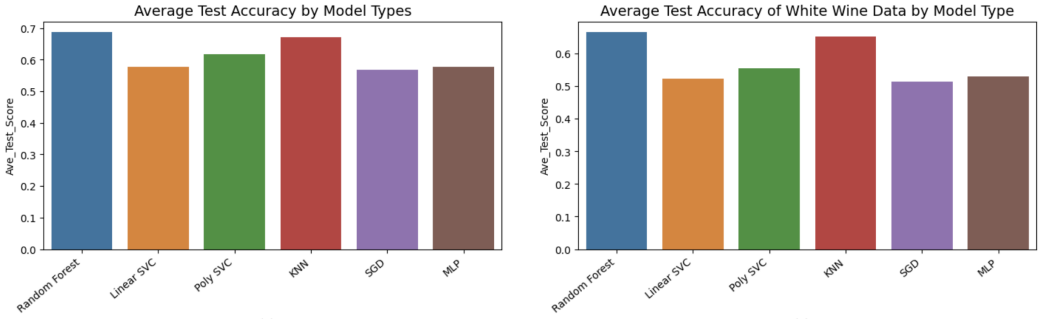
	Model_Type	PreScaled_Test_Score	Min_Test_Score	Ave_Test_Score	Max_Test_Score	Best_Data_Scale
0	Random Forest	0.662512	0.654345	0.665166	0.685174	standard
1	Linear SVC	0.518172	0.520009	0.522731	0.528175	standard
2	Poly SVC	0.535939	0.520214	0.553627	0.596160	standard
3	KNN	0.617597	0.644953	0.652236	0.659249	standard
4	SGD	0.332838	0.484076	0.513408	0.533074	standard
5	MLP	0.460396	0.511432	0.529945	0.548796	standard

While Random Forests were the clear winner in both datasets, K-Nearest Neighbor models performed at a close second. The red wine data subset resulted in the similar performance for both Linear Support Vector Machine and Stochastic Gradient Descent. While white wine data performance remained rather low for on the white wine dataset for most models, red wine data showed improved performance compared to white wine for all model accuracy.

Overall, average performance was best with the Random Forest method across all data scaling but KNN was a close second on all data as shown in the following charts:



Best Results for these models was similar in performance to the more "average performance" results from the prior runs. Red wine data is on the left, and white wine data results are shown on the right:



Future exercises will attempt to further tune hyperparameters beyond those listed by default, though this assured that hyperparameters were consistent throughout model runs.

References

[1] A. Asuncion, D. Newman, UCI Machine Learning Repository, University of California, Irvine (2007). Obtained from https://archive-beta.ics.uci.edu/dataset/186/wine+quality.

[2] C. Harris, K. Millman, S. van der Walt, Array programming with NumPy. Nature 585, 357–362 (2020). DOI: 10.1038/s41586-020-2649-2. https://numpy.org/doc/stable/reference/generated/numpy.isnan.html

[3] M. Waskom, (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021, https://doi.org/10.21105/joss.03021.

[4] Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.